Online Robot Error Detection and Description via Video Temporal Grounding

Anonymous Author(s)
Affiliation
Address
email

Abstract: Detecting and describing failure events during robot manipulation is critical for safe autonomous deployment, yet existing methods either reduce failures to per-frame binary scores or require the complete episode. We formulate robot failure detection as an online video temporal grounding problem, where the model processes streaming video and outputs both the temporal boundaries and a natural-language description of each failure event as it unfolds. Our framework operates chunk-by-chunk with a memory bank over previously observed frames. To bridge the domain gap between general video understanding and manipulation reasoning, we design an automated pipeline that converts robot trajectory datasets into temporal grounding annotations for pre-training, which yields a 73% relative improvement in temporal localization. On our evaluation benchmark, the full model outperforms the strongest online grounding baseline with an mIoU of 0.232 (versus 0.194) and produces failure descriptions with a CIDEr score of 0.854. We release the generated dataset and the human-annotated benchmark to support future research on real-time robot failure detection, description, and recovery.

Keywords: Error Detection, Error Description, Robot Safety

1 Introduction

Reliable detection and description of failure events during robot manipulation is a prerequisite for deploying autonomous systems in safety-critical settings. When a robotic arm drops an object, collides with an obstacle, or executes an incorrect grasp, the downstream control policy must be informed not only *that* a failure occurred, but also *when* it began, *how long* it persisted, and *what* went wrong. Such temporally grounded failure descriptions enable timely recovery, human intervention, and post-hoc diagnosis. Two technical challenges make this problem difficult. First, safety-critical applications require *online* inference, where the detector must operate on an unfinished, streaming video rather than waiting for the full episode to conclude. Second, large vision-language models (VLMs) trained on general video data lack the *domain-specific knowledge* needed to understand manipulation actions and physical interactions, leading to near-zero performance on robotic failure videos without adaptation. Addressing these challenges requires a framework that can perform real-time inference on streaming video while incorporating domain-specific manipulation knowledge.

Despite advances of diverse VLMs, no existing method provides online temporal grounding and natural-language failure descriptions for robot manipulation. The temporal structure inherent in manipulation failures remains largely unexploited as a basis for detection. Methods such as REFLECT [1], AHA [2], and ARMOR [3] generate explanations only at the episode level and cannot localize failures in time. RACER [4] enables online monitoring but relies on single-frame inputs, limiting its ability to capture temporally extended failures. Conversely, temporal grounding methods such as TRACE [5] and VTimeLLM [6] can localize and describe events but require access to the complete video, making them unsuitable for real-time safety monitoring. While OnVTG [7] sup-

Submitted to the 10th Conference on Robot Learning (CoRL 2026). Do not distribute.

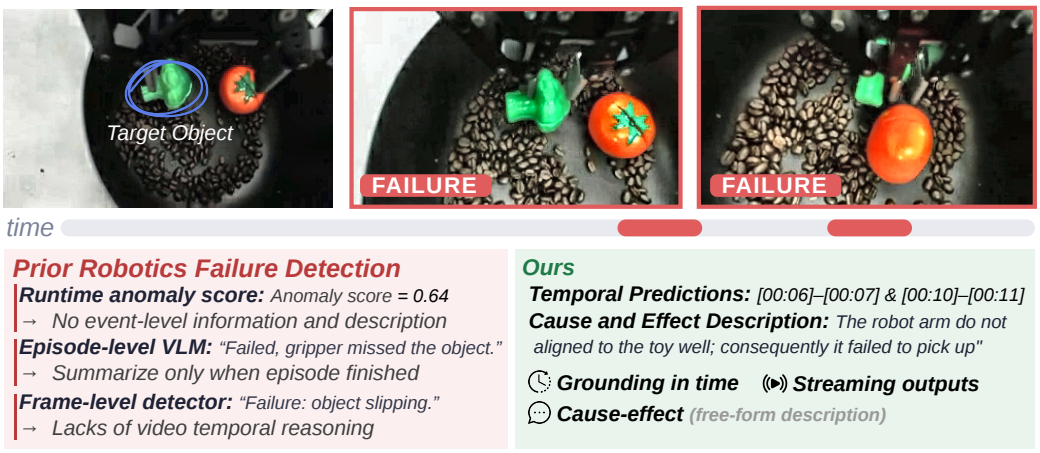

Figure 1: Our online temporal grounding framework detects robot manipulation failures in real time, outputting precise temporal boundaries paired with cause-and-effect descriptions for each failure event as the video streams. Unlike prior approaches (**left**) which are limited to scalar anomaly scores, post-hoc episode summaries, or frame-level labels, our method (**right**) jointly answers *when* a failure occurs and *why*, without waiting for the episode to complete or for offline post-processing.

ports online temporal grounding and VideoLLM-Online [8] enables streaming video understanding, neither provides both temporal localization and natural-language failure descriptions.

A key limitation of existing robot failure detectors is that they often treat failures as instantaneous events, despite typically unfolding as temporal processes. A misaligned grasp, for instance, begins with a subtle offset during approach, escalates when the gripper closes on the wrong surface, and culminates in the object slipping from the hand. To address this limitation, we formulate robot failure detection as an *online video temporal grounding* task, where the model continuously processes streaming video and, upon detecting a failure, predicts both its temporal boundaries and a natural-language description. As illustrated in Fig. 1, this formulation explicitly captures the temporal structure of failures, enabling more precise diagnosis and recovery than the frame-level scores or episode-level summaries that prior detectors have typically relied on.

Building on this formulation, we present the first online temporal grounding framework for robot manipulation failure detection and description. The framework processes video in a chunk-by-chunk manner with a memory bank that aggregates spatial and temporal features from previously observed frames. At each chunk, the model predicts whether a failure event is occurring, estimates its temporal span, and generates a language description of the failure. This combination of temporal localization and failure description provides the diagnostic precision that a recovery policy or human operator requires for timely intervention. Realizing this framework requires temporal grounding annotations for manipulation failures, a form of data that few existing datasets provide. To address this gap, we construct a human-annotated evaluation benchmark, DETECT-DROID-FAILURE, from the DROID dataset [9] and design an automated data generation pipeline that produces temporal grounding annotations from robot trajectory data for pre-training. Pre-training stage improves the mIoU of a Video-LLaMA3 [10] backbone from 0.088 to 0.152, a 73% relative gain, which confirms that the generated annotations effectively bridge the domain gap. On DETECT-DROID-FAILURE, the full online model achieves an mIoU of 0.232 (versus 0.194 for OnVTG) and produces failure descriptions with a CIDEr score of 0.854. Our primary contributions are summarized as follows:

- We formulate manipulation failure detection as an *online video temporal grounding* problem and propose a streaming-memory architecture that processes video chunk-by-chunk, producing both temporal boundaries and natural-language failure descriptions in real time.
- We introduce DETECT-DROID-FAILURE, the first temporally annotated benchmark for robot manipulation failure detection, along with an automated annotation pipeline for generating temporal grounding labels from trajectory data. Both will be publicly released.

2 Related Work

Table 1 summarizes the key dimensions along which our method differs from prior work, which we discuss in detail in the remainder of this section.

2.1 Failure Detection in Robot Manipulation

Table 1: Comparison with related work.

Method	Domain	Video	Online	TG	Free-form	Cause-effect
TRACE	General	✓	✗	✓	✓	✗
OnVTG	General	✓	✓	✓	✗	✗
REFLECT	Robot	▲	✗	✗	✓	✗
AHA	Robot	▲	✗	✗	✓	✗
RACER	Robot	✗	✓	✗	✓	✗
Ours	Robot	✓	✓	✓	✓	✓

Detecting failures during robot manipulation has been studied through two complementary lines of work. The first treats failure detection as a classification or anomaly scoring problem without language descriptions. L-Reformer [11] compares pre- and post-action images to detect state-change anomalies, FAIL-Detect [12] and FIPER [13] cast the problem as out-of-distribution detection, and FINO-Net [14] fuses image, and audio signals for closed-set anomaly classification. While effective at flagging failures, these methods provide no explanation of what went wrong, which limits their utility for recovery planning. The second line incorporates VLMs to generate failure descriptions. REFLECT [1] prompts a VLM to reason about the failure and propose a recovery plan after the episode ends. AHA [2] fine-tunes a VLM on synthetically generated failure data (FailGen) to improve description quality, though it still operates on a small set of sampled frames rather than full video. ARMOR [3] formulates detection and reasoning as a multi-task refinement process. RACER [4] integrates a supervisor VLM into the VLA control loop for frame-level monitoring. Despite rich textual output, these methods operate at either the episode or single-frame level. Few capture the temporal extent of a failure event, which is critical for understanding when the failure began, how it progressed, and what consequences it caused. Our work formulates failure detection as an online temporal grounding task that captures both the temporal boundaries and the natural-language descriptions of each failure event.

2.2 Video Temporal Grounding

Video temporal grounding (VTG) localizes events within a video by predicting start and end timestamps, often accompanied by natural-language descriptions. Offline methods such as VTimeLLM [6], VTG-LLM [15], Grounded-VideoLLM [16], and TRACE [5] process the entire video at once with specialized temporal heads or event-based reformulations. TimeExpert [17] extends TRACE with a mixture-of-experts architecture for multi-task temporal understanding. Nonetheless, these methods require the complete video, which precludes their use in real-time monitoring where the video stream has not yet concluded. Online temporal grounding remains underexplored. OnVTG [7] introduces event proposals with a hierarchical memory for streaming moment retrieval, though it focuses solely on temporal span prediction without generating language descriptions. Online video-language chatbots such as VideoLLM-Online [8] and StreamBridge [18] process video chunks and respond with text, though they lack explicit temporal grounding mechanisms and cannot predict event boundaries. Our work bridges this gap by combining online chunk-by-chunk inference with both temporal span prediction and natural-language failure description generation.

2.3 Data Generation for Robot Learning

Adapting general-purpose vision-language models to the robotics domain requires training data with domain-specific annotations. Robo2VLM [19] constructs a manipulation VQA dataset by decomposing tasks into pre-defined phases and generating question-answer pairs. ECoT [20] follows a similar strategy and generates chain-of-thought VQA data from robot trajectories. AHA [2] proposes FailGen, which synthesizes failure scenarios in simulation for fine-tuning. While these efforts have shown that generated data can bridge the domain gap for manipulation understanding, they focus exclusively on VQA-style supervision. Few provide the temporal grounding annotations (start time, end time, event description) necessary for training a video temporal grounding model. In-

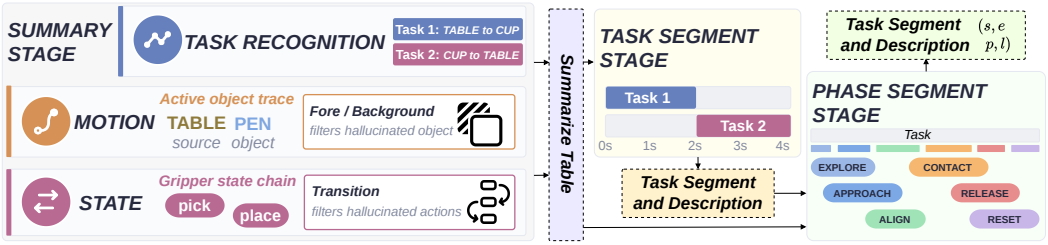

Figure 2: Overview of the architecture of our data generation pipeline. Given a video, the Summary Stage produces a high-level summary that enumerates all tasks, such as grasping and releasing the target object. The Task Segmentation Stage then decomposes the video into temporal task segments, and the Phase Segmentation Stage further divides each task segment into finer-grained subtasks.

spired by dense video captioning pipelines such as VideoMind [21] and VideoEspresso [22], we design an automated data generation pipeline that produces temporal grounding annotations from robot trajectory datasets for pre-training on manipulation-specific failure detection and description.

3 Dataset: DETECT-DROID-FAILURE

Formulating failure detection as online temporal grounding requires supervision in the form of temporal boundary annotations paired with failure descriptions from real robot trajectories. Existing robot failure datasets provide episode-level labels or frame-level anomaly scores, and general-purpose video grounding datasets contain no manipulation content. As a result, neither the training data nor the evaluation benchmark required by this formulation currently exists. To address this gap, we introduce DETECT-DROID-FAILURE, a benchmark built on top of the DROID dataset [9] that contains annotated failure events in real-world robot manipulation trajectories, each labeled with its temporal extent and a natural-language cause.

The DROID dataset contains approximately 16,000 videos of unsuccessful trajectories, the largest publicly available source of real-world robot failure events to our knowledge. For each video v_i, annotators received the corresponding task instruction describing the intended behavior (e.g., "Move object into or out of the container") and identified a set of failure events $\{(s_j, e_j, l_j)\}_{j=0}^{k}$, where s_j and e_j denote the temporal boundaries of the j-th failure event and l_j is its natural-language description. A single video may contain multiple failure events, for example when a robot fails to grasp an object on successive attempts. Each failure event was annotated along two dimensions.

Temporal Boundary Annotation. Annotators labeled the start time s_j and end time e_j of each failure event. The start time s_j was defined as the earliest moment at which the cause of the failure became observable: for a failed grasp of a spoon, this is the moment the gripper makes an unstable contact. The end time e_j was defined as the moment the failure outcome became apparent, such as the spoon slipping from the gripper and falling onto the floor. This temporal definition ensures that each annotated interval captures both the causal process and its observable consequence.

Failure Description Annotation. For each failure event, annotators provided a natural-language description l_j with two components: the failure *outcome*, which describes the observable unsuccessful event, and the failure *cause*, which explains why it occurred from a human-interpretable perspective. For example: **Outcome:** "The spoon slipped out of the gripper and fell onto the floor." **Cause:** "The robot grasped the spoon near its tip, which produced an unstable hold." Annotators were explicitly instructed to separate causes from outcomes so that the annotations support qualitative causal analysis in addition to temporal failure detection. This two-part structure allows trained models to reason about why a failure occurred rather than only detecting that one happened.

4 Methodology

Enabling online robot failure detection with natural-language descriptions requires overcoming two key challenges: no existing architecture jointly performs both tasks in a streaming setting, and

large-scale human-annotated training data for such systems remains scarce. We address these by presenting a unified streaming architecture and an automatic data augmentation pipeline that generates temporally grounded failure descriptions for robot-action videos.

4.1 STREAM-FAIL: Streaming Failure Analysis and Localization

Problem Setting. We consider online robot failure detection and description in a streaming setting. Given a video stream $\{i_t\}_{t=0}^{T}$, a failure is assumed to span from time s to e. Unlike offline methods, the model may only access observations up to the current time step, so when predicting an event ending at e, it operates solely on $\{i_t\}_{t=0}^{e}$. The model has two objectives: (1) temporally localize the failure, and (2) generate a language description of its outcome and cause. Concretely, the model emits a failure-start signal at s and a failure-end signal at e, after which it generates a description l from the accumulated observations $\{i_t\}_{t=s}^{e}$. For each event, the model thus produces a tuple (s, e, l).

Architecture. We formulate online failure duration prediction as a temporal grounding problem, adopting the streaming framework of OnVTG [7], which incrementally predicts event boundaries from incoming video chunks via a multi-scale memory buffer. However, OnVTG is designed solely for temporal localization and cannot generate failure descriptions. We extend it with a language generation module conditioned on the grounded failure interval. Naively coupling a grounding model with a language model is non-trivial, as the latter must be trained to interpret high-dimensional visual representations. We address this by replacing OnVTG's visual backbone with that of VideoLLaMA3 (VL3) [10] and reusing its paired language model, allowing our architecture to inherit VL3's video understanding and language generation capabilities while retaining online temporal grounding through the streaming proposal framework.

Inference. Once the grounding model predicts a failure event (s, e), the corresponding features are retrieved from the memory buffer and passed to the language head to generate the description l, producing the final output tuple (s, e, l). To maintain low-latency online inference, temporal grounding is performed continuously for each incoming chunk, while language generation is executed only after an event is finalized. The grounding and language modules are supervised independently using temporal boundary annotations and failure description annotations, respectively.

4.2 Temporally Grounded Robotics Captioning Data Generation

We design a multi-stage VLM annotation pipeline that converts DROID episodes into temporally grounded robot-action descriptions, providing in-domain supervision that is unavailable in existing public robot datasets. Our goal is to construct a large-scale collection of fine-grained video segments paired with semantically rich descriptions of robot behaviors. As illustrated in Fig. 2, the pipeline consists of three sequential stages. Stage 1 generates a high-level semantic summary describing the overall sequence of actions in the full video. Conditioned on this summary, Stage 2 temporally segments the trajectory into task-level action intervals. Finally, Stage 3 further decomposes each task into fine-grained manipulation phases with corresponding descriptions. To improve annotation reliability, the output of each stage is verified by an additional VLM-based consistency check before being passed to the subsequent stage. The two VLMs are used across the pipeline: Qwen3.5-VL-122B and Cosmos-Reason2-8B. For every stage, the verifier is implemented on the opposite backbone from the generator, so that the two models cross-check each other's outputs. Implementation details of each stage are provided in the supplementary material.

Stage 1: Video Summarization with Grounded Task and Object. Given a robot execution video, the goal of Stage 1 is to generate a high-level summary describing the sequence of tasks performed throughout the trajectory. The summary is represented as an ordered list of temporally sequential tasks. For example, a pick-and-place behavior is often decomposed into two distinct tasks: grasping the object and subsequently releasing it at the target location.

Stage 2: Task-Level Segmentation. Given the task sequence generated in Stage 1, Stage 2 temporally localizes each task within the video and produces a more detailed description for every task segment. Specifically, we prompt the VLM to localize the segment given the corresponding descrip-

tion. Consider a 6 second video in which the robot transports a blue pen between a table and a cup. Given the two task list from Stage 1, Stage 2 emits two temporally disjoint task segments. The first one spanning $[0.0,\ 3.0]$ s, and the description *pick up the blue pen from the table and place it in the green cup*. The second one spanning $[3.0,\ 6.0]$ s, describes the another motion, *pick the blue pen out of the green cup and return it to the table*. Each window covers a full manipulation cycle.

Stage 3: Phase-Level Segmentation. Given a single-task clip produced by Stage 2, Stage 3 further segments it into a sequence of manipulation phases, and also describes each phase. The phase taxonomy is defined as the structure of a manipulation cycle, exploring, approaching, aligning, contacting, releasing, and resetting. Consequently, we obtain a set of $\{(s, e, l, p)\}$, where s, e indicate the start and end of the phase, p indicates the phase category, and l indicates their description. For the first task from Stage 2, a representative set of $\{(s, e, l, p)\}$ are $(1.0\ , 1.5\ ,$ the gripper closes around the pen and lifts it off the table, contact$)$. Applied to the full DROID dataset, our pipeline processes 95K robot videos and outputs on average 1.8 tasks per video and 5.61 phases per task, yielding approximately 950K phase tuples (s, e, l, p) in total.

Model Training. To leverage the generated data, we employ a two-stage training strategy. We first pre-train the VLM on the synthetic dataset produced by our data generation pipeline along with part of the human-annotated DETECT-DROID-FAILURE dataset, allowing the model to learn robot-domain concepts at scale. The model is subsequently fine-tuned on the human-annotated DETECT-DROID-FAILURE dataset to obtain the final detector.

5 Experimental Results

5.1 Experimental Setup

We evaluate on DETECT-DROID-FAILURE, the temporally annotated benchmark of Section 3, whose episodes are real-world DROID [9] manipulation trajectories in which each failure carries a triplet (s, e, l) for its onset, termination, and cause. Our model processes each episode as a stream and conditions only on past frames, as safety monitoring requires. Several baselines instead require access to the complete episode, a less restrictive setting than our strictly online evaluation protocol.

The evaluation targets the two capabilities our task couples and that prior benchmarks have assessed only in isolation. **Temporal localization** is reported as the mean temporal intersection-over-union (mIoU) between predicted and ground-truth intervals, together with F1 at IoU thresholds of 0.3, 0.5, and 0.7. A prediction counts as a true positive once its temporal overlap with a ground-truth event exceeds the threshold. For the frame-wise baselines, which emit a per-frame anomaly score rather than intervals, we report Hit-rate, the fraction of test videos for which the peak anomaly score frame falls inside any ground-truth failure interval, and the Mean distance, the mean seconds between this anomaly frame and the nearest edge of an interval. **Description quality** is measured only on events that are first localized correctly at an IoU of 0.7, which prevents a fluent description of a mistimed or spurious detection from inflating the score. On these events we report CIDEr [23] against the reference descriptions, together with an LLM-as-judge [24] rating of semantic correctness and faithfulness. This coupling ensures strong performance requires both accurate localization and faithful description.

We benchmark against OnVTG [7], the only prior streaming temporal grounding method and thus our primary online baseline, although it emits temporal spans without descriptions. We further compare against representative robot failure methods that each cover only part of the task: the framewise anomaly detectors FINO-Net [14] and MGFN [25] for localization, and the episode-level describer AHA [2] for failure description. To evaluate whether in-domain pretraining helps regardless of the backbone, we additionally report zero-shot and pretrained variants of three video-language backbones (Table 3). Full implementation details are provided in the supplementary material.

5.2 Overview of Results

Table 2 contrasts STREAM-FAIL with prior robot failure methods, each designed for one side of the task. FINO-Net and MGFN are anomaly scorers that flag when execution leaves the normal regime. They address localization alone, and even there their frame-level scores place an event coarsely. AHA, an episode-level describer, explains a completed episode in language and therefore addresses description alone, without temporal extent. STREAM-FAIL addresses both sides because its architecture couples a streaming grounding framework with a language head conditioned on the predicted interval (Section 4). Within a single online pass it locates a failure to within roughly a second of its onset and generates a description scored against event-level references, the joint capability that Table 1 attributes to no prior method.

Table 2: Comparison with prior robot failure methods on DETECT-DROID-FAILURE. FINO-Net and MGFN apply to localization only, scored by hit-rate and mean temporal distance, whereas AHA emits a single global description. STREAM-FAIL is evaluated on both the localization and description axes.

	Localization		Description	
Methods	Hit-rate ↑	Mean dist ↓	CIDEr@0.7 ↑	LLM ↑
FINO-Net [14]	0.007	8.34	-	-
MGFN [25]	0.068	6.77	-	-
AHA [2]	-	-	0.000	1.21
STREAM-FAIL	**0.426**	**1.48**	**0.854**	**2.56**

5.3 Effect of Generated Data on Pretraining and Downstream Fine-tuning

The central premise behind our data-generation pipeline is that general video-language models underperform on robot failure analysis for lack of in-domain supervision, and that this supervision can be produced automatically instead of annotated by hand. Table 3 tests that premise by pretraining each backbone on the generated corpus and evaluating it before any fine-tuning on the human benchmark. Without this pretraining, a general video-language model is close to unusable on the task, with weak localization and description quality near zero, the domain gap the introduction identifies for off-the-shelf VLMs. Pretraining on the generated data closes much of this gap. The effect is clearest on VL3-7B, the backbone we deploy, where localization mIoU improves by roughly 70% and description quality rises from near zero, while the other backbones gain mainly in localization. The pipeline provides the manipulation-specific supervision these models lack.

Table 3: Effect of pretraining on the generated corpus across three video-language backbones, evaluated before any fine-tuning on DETECT-DROID-FAILURE. Each backbone is reported without and with (✓) the pretraining. The gain is largest for VL3-7B, the backbone we adopt for the full model.

		Localization			Description	
Backbone	Pre-trained	mIoU	F1@0.7	F1@0.3	CIDEr@0.7	LLM
VL3-2B		0.0464	0.0033	0.0845	0.000	1.71
	✓	0.1140	0.0121	0.0551	0.1902	1.67
VL3-7B		0.0877	0.0106	**0.2430**	0.000	2.03
	✓	**0.1522**	**0.1113**	0.1910	**0.745**	**2.65**
Cosmos		0.1265	0.0085	0.2134	0.004	1.65
	✓	0.1192	0.0091	0.2231	0.032	**2.72**

This benefit is not confined to the zero-shot setting. Even with the human benchmark available for fine-tuning, mixing the generated corpus with part of DETECT-DROID-FAILURE as a pretraining stage improves every metric in Table 4, with the clearest gain in description quality. The two sources prove complementary because they supply different supervision: human annotation contributes precise event boundaries and reference wording defining the benchmark, whereas the generated corpus contributes the same supervision at a scale manual effort cannot match. The deployment model's competence is therefore not bounded by the human-labeled set, and much comes from unlabeled trajectories at low cost.

Table 4: Effect of the generated corpus on downstream fine-tuning. VL3-7B is fine-tuned on DETECT-DROID-FAILURE with and without (✓) the corpus as prior pretraining. Adding it improves every metric, with the largest gains concentrated in description quality.

Generated data	Localization			Description	
	mIoU	F1@0.7	F1@0.3	CIDEr@0.7	LLM
	0.2365	0.1833	0.3040	1.006	3.16
✓	**0.2471**	**0.1990**	**0.3082**	**1.182**	**3.19**

5.4 Analysis of the Architecture

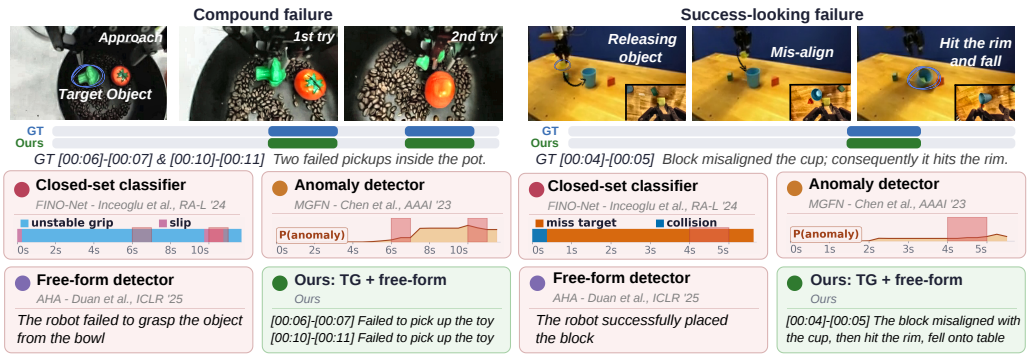

Figure 3: Qualitative comparison on two challenging episodes from DETECT-DROID-FAILURE. **Left:** a compound failure with two consecutive failed pickups. **Right:** a success-looking failure in which the block misaligns with the cup and strikes its rim. Each panel shows the ground-truth intervals alongside an anomaly detector, a free-form describer, and STREAM-FAIL. Only STREAM-FAIL recovers the failure boundaries and pairs each with a cause-and-effect description.

Table 5 compares STREAM-FAIL with OnVTG, the streaming grounding framework it builds on, under the two changes we introduce: the visual backbone is replaced with that of VL3, and a language head is added. Both models are fine-tuned on identical data, and architecture is the only difference. Localization does not suffer from these changes: mean IoU is in fact higher than OnVTG, while F1 at the strictest threshold stays within a small margin. The changes add what OnVTG cannot provide, a description for each failure. The architecture gains explanation at no cost to localization accuracy.

Table 5: Architecture comparison between OnVTG and STREAM-FAIL, both fine-tuned on DETECT-DROID-FAILURE for a controlled comparison. Replacing the specialized backbone with a video-language model and adding a language head preserves localization while adding the description OnVTG lacks.

	Localization		Description	
Architecture	mIoU ↑	F1@0.7 ↑	CIDEr@0.7 ↑	LLM ↑
OnVTG	0.1941	**0.1411**	-	-
STREAM-FAIL	**0.2315**	0.1365	**0.8544**	**2.56**

5.5 Qualitative Result

Fig. 3 compares STREAM-FAIL with three classes of prior work on two failures. On the compound failure, the baselines cannot separate the two failed pickups: they smear a single anomaly score across both or collapse them into one summary, whereas STREAM-FAIL emits a distinct tuple per attempt. On the success-looking failure, the baselines report no failure at all, whereas STREAM-FAIL localizes the misalignment and explains it.

6 Conclusion

In this paper, we formulated online robot failure detection as online video temporal grounding and introduced STREAM-FAIL, which localizes and describes failures in a single streaming pass. To support the task, we built the DETECT-DROID-FAILURE benchmark and an automatic pipeline that generates grounded failure descriptions from unlabeled trajectories. Experiments showed that STREAM-FAIL retained strong localization while adding explanation, and that the generated supervision improved robot-domain video-language understanding.

7 Limitations

Two limitations bound the present study. The evaluation covers real-world manipulation trajectories from DROID, and extending the approach to other embodiments, longer-horizon tasks, and non-manipulation settings remains future work. The detector is also studied as a standalone monitor, and integrating its outputs into a recovery or vision-language-action policy, the deployment scenario that motivates this work, is a natural and important direction for future work.

References

[1] Z. Liu, A. Bahety, and S. Song. REFLECT: Summarizing robot experiences for failure explanation and correction. In *Proceedings of the 7th Conference on Robot Learning (CoRL)*, 2023.

[2] J. Duan, W. Pumacay, N. Kumar, Y. R. Wang, S. Tian, W. Yuan, R. Krishna, D. Fox, A. Mandlekar, and Y. Guo. AHA: A vision-language-model for detecting and reasoning over failures in robotic manipulation. In *Proceedings of the 13th International Conference on Learning Representations (ICLR)*, 2025.

[3] C. Qi, X. Wang, S. Yong, S. Sheng, H. Mao, sriram srinivasan, M. Nambi, A. Zhang, and Y. Dattatreya. Self-refining vision language model for robotic failure detection and reasoning. In *The Fourteenth International Conference on Learning Representations*, 2026. URL https://openreview.net/forum?id=jr9hGWQioP.

[4] Y. Dai, J. Lee, N. Fazeli, and J. Chai. RACER: Rich language-guided failure recovery policies for imitation learning. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2025.

[5] Y. Guo, J. Liu, M. Li, Q. Liu, X. Chen, and X. Tang. TRACE: Temporal grounding video LLM via causal event modeling. In *The Thirteenth International Conference on Learning Representations*, 2025. URL https://openreview.net/forum?id=14fFV0chUS.

[6] B. Huang, X. Wang, H. Chen, Z. Song, and W. Zhu. VTimeLLM: Empower LLM to grasp video moments. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 14271–14280, 2024.

[7] M. Zheng, Y. Peng, B. Sun, Y. Yang, and Y. Liu. Hierarchical event memory for accurate and low-latency online video temporal grounding. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 21589–21599, October 2025.

[8] J. Chen, Z. Lv, S. Wu, K. Q. Lin, C. Song, D. Gao, J.-W. Liu, Z. Gao, D. Mao, and M. Z. Shou. VideoLLM-online: Online video large language model for streaming video. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.

[9] A. Khazatsky, K. Pertsch, S. Nair, A. Balakrishna, S. Dasari, S. Karamcheti, S. Nasiriany, M. K. Srirama, L. Y. Chen, K. Ellis, P. D. Fagan, J. Hejna, M. Itkina, M. Lepert, Y. J. Ma, P. T. Miller, J. Wu, S. Belkhale, S. Dass, H. Ha, A. Jain, A. Lee, Y. Lee, M. Memmel, S. Park, I. Radosavovic, K. Wang, A. Zhan, K. Black, C. Chi, K. B. Hatch, S. Lin, J. Lu, J. Mercat, A. Rehman, P. R. Sanketi, A. Sharma, C. Simpson, Q. Vuong, H. R. Walke, B. Wulfe, T. Xiao, J. H. Yang, A. Yavary, T. Z. Zhao, C. Agia, R. Baijal, M. G. Castro, D. Chen, Q. Chen, T. Chung, J. Drake, E. P. Foster, J. Gao, V. Guizilini, D. A. Herrera, M. Heo, K. Hsu, J. Hu, M. Z. Irshad, D. Jackson, C. Le, Y. Li, K. Lin, R. Lin, Z. Ma, A. Maddukuri, S. Mirchandani, D. Morton, T. Nguyen, A. O'Neill, R. Scalise, D. Seale, V. Son, S. Tian, E. Tran, A. E. Wang, Y. Wu, A. Xie, J. Yang, P. Yin, Y. Zhang, O. Bastani, G. Berseth, J. Bohg, K. Goldberg, A. Gupta, A. Gupta, D. Jayaraman, J. J. Lim, J. Malik, R. Martín-Martín, S. Ramamoorthy, D. Sadigh, S. Song, J. Wu, M. C. Yip, Y. Zhu, T. Kollar, S. Levine, and C. Finn. Droid: A large-scale in-the-wild robot manipulation dataset. 2024.

[10] B. Zhang, K. Li, Z. Cheng, Z. Hu, Y. Yuan, G. Chen, S. Leng, Y. Jiang, H. Zhang, X. Li, et al. Videollama 3: Frontier multimodal foundation models for image and video understanding. *arXiv preprint arXiv:2501.13106*, 2025.

[11] M. Goko, M. Kambara, D. Saito, S. Otsuki, and K. Sugiura. Task success prediction for open-vocabulary manipulation based on multi-level aligned representations. In *Proceedings of the 8th Conference on Robot Learning (CoRL)*, 2024.

[12] C. Xu, T. K. Nguyen, E. Dixon, C. Rodriguez, P. Miller, R. Lee, P. Shah, R. Ambrus, H. Nishimura, and M. Itkina. Can we detect failures without failure data? Uncertainty-Aware runtime failure detection for imitation learning policies. In *RSS 2025 workshop Robot Evaluation for the Real World*, 2025. URL https://openreview.net/forum?id=A2iUXYdWZD.

[13] R. Römer, A. Kobras, L. Worbis, and A. Schoellig. Failure prediction at runtime for generative robot policies. *Advances in Neural Information Processing Systems*, 38:7631–7670, 2025.

[14] A. Inceoglu, E. E. Aksoy, and S. Sariel. Multimodal detection and classification of robot manipulation failures. *IEEE Robotics and Automation Letters*, 9(2):1396–1403, 2024.

[15] Y. Guo, J. Liu, M. Li, D. Cheng, X. Tang, D. Sui, Q. Liu, X. Chen, and K. Zhao. Vtg-llm: Integrating timestamp knowledge into video llms for enhanced video temporal grounding. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, pages 3302–3310, 2025.

[16] H. Wang, Z. Xu, Y. Cheng, S. Diao, Y. Zhou, Y. Cao, Q. Wang, W. Ge, and L. Huang. Grounded-VideoLLM: Sharpening fine-grained temporal grounding in video large language models. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, 2025.

[17] Z. Yang, Y. Yu, Y. Zhao, S. Lu, and S. Bai. TimeExpert: An expert-guided video LLM for video temporal grounding. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 24286–24296, 2025.

[18] H. Wang, B. Feng, Z. Lai, M. Xu, S. Li, W. Ge, A. Dehghan, M. Cao, and P. Huang. StreamBridge: Turning your offline video large language model into a proactive streaming assistant. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.

[19] K. Chen, S. Xie, Z. Ma, P. R. Sanketi, and K. Goldberg. Robo2VLM: Improving visual question answering using large-scale robot manipulation data. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems Datasets and Benchmarks Track*, 2025. URL https://openreview.net/forum?id=OChorZcZnY.

[20] M. Zawalski, W. Chen, K. Pertsch, O. Mees, C. Finn, and S. Levine. Robotic control via embodied chain-of-thought reasoning. In *Conference on Robot Learning*, pages 3157–3181. PMLR, 2025.

[21] Y. Liu, K. Q. Lin, C. W. Chen, and M. Z. Shou. Videomind: A chain-of-loRA agent for temporal-grounded video reasoning. In *The Fourteenth International Conference on Learning Representations*, 2026. URL https://openreview.net/forum?id=57EwidOnSf.

[22] S. Han, W. Huang, H. Shi, L. Zhuo, X. Su, S. Zhang, X. Zhou, X. Qi, Y. Liao, and S. Liu. Videoespresso: A large-scale chain-of-thought dataset for fine-grained video reasoning via core frame selection. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 26181–26191, June 2025.

[23] R. Vedantam, C. Lawrence Zitnick, and D. Parikh. Cider: Consensus-based image description evaluation. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 4566–4575, 2015.

[24] J. Gu, X. Jiang, Z. Shi, H. Tan, X. Zhai, C. Xu, W. Li, Y. Shen, S. Ma, H. Liu, et al. A survey on llm-as-a-judge. *The Innovation*, 2024.

[25] Y. Chen, Z. Liu, B. Zhang, W. Fok, X. Qi, and Y.-C. Wu. Mgfn: Magnitude-contrastive glance-and-focus network for weakly-supervised video anomaly detection. In *Proceedings of the AAAI conference on artificial intelligence*, volume 37, pages 387–395, 2023.